

SHELL Programmierung

Basics, WH

Susanne Schaller, MMSc - susanne.schaller@fh-hagenberg.at

Andreas Scheibenpflug, MSc - andreas.scheibenpflug@fh-hagenberg.at



Was kann die Shell? (1)

- Shell ist ein Kommandointerpreter
 - Sh, csh, ksh, bash oder ein selbst geschriebener Kommandozeileninterpreter
- Kommandos entgegennehmen
- Kommando suchen
- Ein- und Ausgabe des Kommandos im Shellfenster durchführen
- Ersetzen von Sonderzeichen und Befehlssubstitution
- Ablaufsteuerung (Anweisungen, Schleifen, etc. → Shellprogrammierung)

Was kann die Shell? (2)

- Shell parst die Argumente eines Kommandos
- Shell Variablen werden durch ihre Werte entsprechend ersetzt
- Kommandosubstitution wird durchgeführt
- Stdin, stdout, stderr wird entsprechend umgeleitet mit 0,1,2
- Variablenzuweisungen werden durchgeführt
- Wildcards werden ersetzt
- Shell führt das Kommando aus

Ein- / Ausgabeumleitung (1)

- Eingabeumleitung oft können Befehle aber ohnehin mit Dateien umgehen
 - Programm liest aus Datei, nicht von Tastatur
 - Umleitung erfolgt durch ,<‘
 - Numerischer Wert: 0
- Ausgabeumleitung
 - Ergebnis wird in Datei geschrieben, nicht auf Bildschirm
 - Umleitung erfolgt durch ,>‘ bzw. ,>>‘
 - Numerischer Wert: 1



append, d.h. nicht überschreiben
sondern anfügen

Ein- / Ausgabeumleitung (2)

- Umlenken der Fehlerausgabe
 - NUR Fehlermeldungen werde in Datei geschrieben
 - Umleitung erfolgt durch `,2>'`
 - Numerischer Wert: 2
- Spezialziel
 - `/dev/null` „Schwarzes Loch“, „Nirvana“
 - Beispiel: `ls > /dev/null/` - Ausgabe verschwindet gänzlich

Beispiele

```
osboxes@osboxes:~/Desktop$ ls -l > myfile.dat
osboxes@osboxes:~/Desktop$ ls -l >> myfile.dat
osboxes@osboxes:~/Desktop$ cat < input.dat
```

```
osboxes@osboxes: ~/Desktop
osboxes@osboxes:~/Desktop$ wc < myfile.dat > output_file.txt
osboxes@osboxes:~/Desktop$ cat output_file.txt
```

```
52  454 2868
osboxes@osboxes:~/Desktop$ wc < myfile.dat
52  454 2868
```

```
osboxes@osboxes:~/Desktop$ ls -l > myfile.dat 2> errors.err
osboxes@osboxes:~/Desktop$ cat errors.err
osboxes@osboxes:~/Desktop$ nano errors.err
osboxes@osboxes:~/Desktop$ ls -l > myfile.dat 2> /dev/null
osboxes@osboxes:~/Desktop$ ls -l 2>&1
total 80
-rw-rw-r-- 1 osboxes osboxes 722 Dec 20 12:50 abfahrt.txt
-rw-rw-r-- 1 osboxes osboxes  0 Apr 28 2016 abfahrt.txt~
-rw-rw-r-- 1 osboxes osboxes  0 Jun 10 2016 besucher.txt~
-rw-rw-r-- 1 osboxes osboxes 785 Jan 10 09:51 chicagoBulls
-rw-rw-r-- 1 osboxes osboxes 785 Oct 17 08:08 chicagoBulls~
-rw-rw-r-- 1 osboxes osboxes 786 Jan 10 09:41 chicagobulls.txt
```

stderr auf
stdout
umlenken

Kommando echo

- Befehl echo erzeugt Text am Bildschirm
- Dafür wird er auch in Shell-Skripts verwendet
 - echo Hello World → erzeugt die Ausgabe Hello World

```
osboxes@osboxes:~/Desktop$ echo echo Hallo, ich bin dein zweites Skript! > skript2
osboxes@osboxes:~/Desktop$ ./skript2
bash: ./skript2: Permission denied
osboxes@osboxes:~/Desktop$ ls -l skript2
-rw-rw-r-- 1 osboxes osboxes 41 Jan 12 15:12 skript2
osboxes@osboxes:~/Desktop$ chmod u+x skript2
osboxes@osboxes:~/Desktop$ ./skript2
Hallo, ich bin dein zweites Skript!
```

Pipes

- Eine Pipe verbindet Ausgabe eines Programms und Eingabe eines zweiten Programms über einen temporären Puffer
- Mehrere Kommandos können hintereinander geschaltet werden

```
osboxes@osboxes: ~/Desktop$ dmesg | grep usb
```

osboxes@osboxes:~/Desktop\$ dmesg | grep usb

```
[ 3.399409] usbcore: registered new interface driver usbfs
[ 3.399414] usbcore: registered new interface driver hub
[ 3.399419] usbcore: registered new device driver usb
[ 3.801842] usb usb1: New USB device found, idVendor=1d6b, idProduct=
[ 3.801852] usb usb1: New USB device strings: Mfr=3, Product=2, Serial
[ 3.801857] usb usb1: Product: OHCI PCI host controller
[ 3.801862] usb usb1: Manufacturer: Linux 4.2.0-16-generic ohci_hcd
[ 3.801865] usb usb1: SerialNumber: 0000:00:06.0
[ 4.480333] usb 1-1: new full-speed USB device number 2 using ohci-pc
[ 4.736482] usb 1-1: New USB device found, idVendor=80ee, idProduct=0021
[ 4.736491] usb 1-1: New USB device strings: Mfr=1, Product=3, SerialNumber=0
[ 4.736497] usb 1-1: Product: USB Tablet
[ 4.736501] usb 1-1: Manufacturer: VirtualBox
[ 4.762817] usbcore: registered new interface driver usbhid
[ 4.762825] usbhid: USB HID core driver
[ 4.768414] input: VirtualBox USB Tablet as /devices/pci0000:00/0000:00:06.0/usb1/1-1/1-1:1.
0/0003:80EE:0021.0001/input/input6
[ 4.768646] hid-generic 0003:80EE:0021.0001: input,hidraw0: USB HID v1.10 Mouse [VirtualBox
USB Tablet] on usb-0000:00:06.0-1/input0
osboxes@osboxes:~/Desktop$
```

NAME dmesg - print or control the kernel ring buffer

SYNOPSIS dmesg [options]

dmesg --clear
dmesg --read-clear [options]
dmesg --console-level level
dmesg --console-on
dmesg --console-off

Vorgehensweise bei Pipes

- Kombination aus den Befehlen `tr` und `cut`
 - Wird verwendet, wenn aus einem String (Zeichenkette), ein Teil(-String) extrahiert werden muss
- Beispiel: Ermitteln der Größe einer Datei (`/etc/passwd`) – `ls -l /etc/passwd`
 - Ergebnis sind Angaben zur Datei `/etc/passwd`.
 - Angaben sind in Spalten (Feldern) angeordnet.
 - Felder werden durch eine unterschiedliche Anzahl von Leerzeichen getrennt.
 - Leerzeichen zwischen den einzelnen Feldern sind bis auf jeweils eins zu reduzieren.
 - Dazu den Befehl `tr -s " "` verwenden (**man tr**)
 - Die Standard-Ausgabe des Befehls `ls` wird mit Hilfe des `|` (Pipe-Operator) auf die Standard-Eingabe des Befehls `tr` umgeleitet.

```
ls -l /etc/passwd | tr -s " "
-rw-r--r-- 1 root root 1530 Aug 24 14:07 /etc/passwd
```

Vorgehensweise bei Pipes

- Jetzt muss nur noch die Dateigrösse ermittelt werden.
- Mit dem Befehl **cut** kann man das betreffende Feld ausschneiden.
- Hierzu folgende 2 Optionen verwenden. (man cut)
 - d " " bezeichnet das Trennzeichen zwischen den einzelnen Feldern
 - f 5 bezeichnet das betreffende Feld (Feld 5 enthält Dateigrösse)
- Das vorhergehende Ergebnis wird wieder mit Hilfe des Piping in die Standard- Eingabe des Befehls **cut** umgeleitet.
- Endgültiges Kommando:

```
ls -l /etc/passwd | tr -s " " | cut -d " " -f 5
```

Ausgabe: 1530

Wildcards

Wildcard	Beschreibung
?	Ersetzt ein beliebiges Zeichen.
*	Ersetzt beliebig viele oder Null Zeichen und ist gleichzeitig die Wildcard für alle regulären Dateien. Die Dateien dürfen nicht mit einem Punkt "." beginnen, allerdings dürfen sie ab dem 2. Zeichen beliebig viele Punkte enthalten.
xyz*	Beginnt mit xyz, danach beliebig viele Zeichen.
.*	Alle versteckten Dateien, d.h. die Datei beginnt mit einem Punkt.
{*,.[!]*}	Diese Kombination aus Klammererweiterung und Wildcards berücksichtigt alle regulären und alle versteckten Dateien und verhindert gleichzeitig, dass der Vorgang auf höheren Verzeichnisebenen übergreift. Im unten stehenden Beispiel 2, wird auf diese Problematik noch einmal genauer eingegangen.
.	Die Datei muss mindestens einen Punkt enthalten, kann aber auch mehrere Punkte enthalten.
abc*.*	Beginnt mit abc, danach mindestens ein Punkt.
[abc]	In den eckigen Klammern stehen Platzhalter für genau ein Zeichen, hier a, b oder c.
[o-u]	w.v., genau ein Zeichen, in diesem Fall aus dem Bereich der Kleinbuchstaben, von o-u.
[A-Z]	w.v., hier alle Grossbuchstaben von A-Z. Damit sind dann Kleinbuchstaben, Zahlen und alle sonstigen Zeichen ausgeschlossen.
[A-Z]7	A7 B7 ... Z7
[A-Za-z] dat	Adat Bdat ... Zdat adat bdat ... zdat
[!x]	Nicht x.
[^x]	Nicht x.
[!abc]	Keines der Zeichen aus dem Zeichenvorrat in der eckigen Klammer.
[abc]de	Kann sein: ade bde cde.
~	Joker für das Home-Verzeichnis.

Globbering - Pfadnamenerweiterung

- Um an ein Kommando eine Liste von Dateinamen zur Verarbeitung zu übergeben, können diese vollständig angegeben werden. Oder man gibt ein oder mehrere „Suchmuster“ (Wildcards) an, die von der aktuellen Shell in passende Dateinamen umgewandelt werden.
- Die Shell expandiert zuerst alle Suchmuster zu einer Liste von passenden Dateinamen ("filename globbing"), setzt diese dann anstelle der Suchmuster ein und führt schließlich das Kommando aus.
- Unabhängig vom auszuführenden Kommando wird die Dateinamenexpansion immer von der Shell durchgeführt, BEVOR ein Kommando gestartet wird.

Globbering – Pitfalls

1. Versuch

```
osboxes@osboxes: ~/Desktop
osboxes@osboxes:~/Desktop$ ls
abfahrt.txt      chicagoBulls~  DIR3}          myfile.dat      script.sh      temp.txt      ue1
abfahrt.txt~     chicagobulls.txt  errors.err     output_file.txt  skript2        temp.txt~     Uebung3
beisucher.txt~   chicagoBullsy     example~       positions       SS16           tester.sh     ~Uebung3
chicagoBulls    curr.txt~       file_ue        raum.txt~       stuff          tester.txt    WS1617
osboxes@osboxes:~/Desktop$ mv *.txt *.dat
mv: target 'myfile.dat' is not a directory
```

MV(1)	User Commands	MV(1)
NAME		
mv - move (rename) files		
SYNOPSIS		
mv [OPTION]... [-I] SOURCE DEST		
mv [OPTION]... SOURCE... DIRECTORY		
mv [OPTION]... -t DIRECTORY SOURCE...		
DESCRIPTION		
Rename SOURCE to DEST, or move SOURCE(s) to DIRECTORY.		

Wäre nur eine Textdatei vorhanden, wird sie in *.dat umbenannt.

Wildcards – Beispiele und Pitfalls

2. Versuch

2. Versuch

```
for f in *.txt
do
    mv ./"$f" "${f%.txt}.bat"
done
```

Erstellung eines Shellskripts –
Shellvariablen darauf
zugreifen und genaue
Definition – ok!

Das würde das gewünschte
Ergebnis liefern.

Shell Variablen (1)

- Möglichkeit, Variablen Werte (Zeichenketten) zu zuweisen
- Gültigkeit:
 - Ab Zuweisung ist Variable gültig
 - Zuweisung: `name=wert`
 - Gilt nur in aktueller Shell
 - Zugriff in anderen Shells mit Kommando `export` kann ermöglicht werden:
 - `export name`

Shell Variablen (2)

- Verwendung:
 - `set` listet alle bereits definierten Variablen auf
 - Zugriff über `$nameofVariable` (Beispiel `echo $PATH`)
 - Falls Variable noch nicht gesetzt, wird leere Zeichenkette eingesetzt
- Umgebungsvariablen
 - Spezielle Bedeutung
 - Von Programmen als vorhanden vorausgesetzt

Beispiel Shell Variablen

```
osboxes@osboxes:~/Desktop$ s=12
osboxes@osboxes:~/Desktop$ echo s
s
osboxes@osboxes:~/Desktop$ echo "$s"
12
osboxes@osboxes:~/Desktop$ echo $s
12
osboxes@osboxes:~/Desktop$ export s
osboxes@osboxes:~/Desktop$ set
BASH=/bin/bash
BASHOPTS=checkwinsize:cmdhist:complete_fullquote:expand_aliases:extglob:extquote:force_ignore:
histappend:interactive_comments:progcomp:promptvars:sourcepath
BASH_ALIASES=()
BASH_ARGC=()
BASH_ARGV=()
BASH_CMDS=()
BASH_COMPLETION_COMPAT_DIR=/etc/bash_completion.d
BASH_LINENO=()
BASH_REMATCH=()
BASH_SOURCE=()
BASH_VERSINFO=([0]="4" [1]="3" [2]="42" [3]="1" [4]="release" [5]="x86_64-pc-linux-gnu")
BASH_VERSION='4.3.42(1)-release'
CLUTTER_IM_MODULE=xim
COLUMNS=95
COMPIZ_BIN_PATH=/usr/bin/
COMPIZ_CONFIG_PROFILE=ubuntu
COMP_WORDBREAKS=$' \t\n\"'><=;|&(:'
```

Shell Umgebungsvariablen

```
osboxes@osboxes:~/Desktop$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games
osboxes@osboxes:~/Desktop$ echo $PS1
\[ \e]0;\u@\h: \w\a\]${debian_chroot:+($debian_chroot)}\[ \033[01;32m\]\u@\h\[ \033[00m\]:\[ \033[01;34m\]\w\[ \033[00m\]\$
osboxes@osboxes:~/Desktop$ echo $PS2
>
osboxes@osboxes:~/Desktop$ echo $HOME
/home/osboxes
osboxes@osboxes:~/Desktop$ echo $HOSTNAME
osboxes
osboxes@osboxes:~/Desktop$ echo $HOSTTYPE
x86_64
osboxes@osboxes:~/Desktop$ echo $PWD
/home/osboxes/Desktop
osboxes@osboxes:~/Desktop$ echo $USER
osboxes
```

PATH	Liste der Verzeichnisse, wo nach Kommandos gesucht wird
PS1 und PS2	Aussehen des Prompts
HOME	Home-Verzeichnis
HOSTNAME	Name des Rechners
HOSTTYPE	Typ des Rechners
PWD	aktuelles Verzeichnis
USER	Loginname des Benutzers
u.v.a.m	

Beispiel – Ändern des Pfades

- Die Konfiguration der PATH-Variable wird in /etc/profile vorgenommen
- Die einzelnen Pfade werden durch „:“ getrennt

```
osboxes@osboxes:~/Desktop$ echo $PATH  
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games
```

- Wenn man die Variable um eine Pfadangabe erweitern will, verwendet man diesen Ausdruck:
 - PATH=\$PATH:/home/userxyz/bin
 - export PATH
- Aktuelles Arbeitsverzeichnis in Pfad einfügen
 - PATH=\$PATH:.
 - export PATH

Variablen werden mit {} eingegrenzt
d.h. \${VAR} -- variable

\$(Befehl) -- führt Befehl aus und gibt Ausgabe
zurück

\$((Rechnung)) -- rechnet rechnung aus und liefert Ergebnis

Quoting

- Verschiedene Hochkomma haben Spezialbedeutung
 - `".. "` Keine Ersetzung der (`*`, `?`, `[]`) Wildcards, jedoch SHELL-Variable mit `$` werden ersetzt
 - `'... '` unterdrückt jede Substitution
 - ``...`` Kommando zwischen Back-Quotes, wird ausgeführt und Ergebnis liefert Wert der Variablen oder `$(Befehl)`

```
osboxes@osboxes: ~/Desktop
osboxes@osboxes:~/Desktop$ VAR="Hello World!"
osboxes@osboxes:~/Desktop$ echo $VAR
Hello World!
osboxes@osboxes:~/Desktop$ echo '$VAR'
$VAR
osboxes@osboxes:~/Desktop$ VAR=`pwd`
osboxes@osboxes:~/Desktop$ echo $VAR
/home/osboxes/Desktop
osboxes@osboxes:~/Desktop$ echo ${VAR}/bin
/home/osboxes/Desktop/bin
```